# PROJECT REPORT

## TITLE：Customer Registry Management System

# 1. INTRODUCTION

## 1.1 Project Overview

The Customer Registry Management System is a full-stack web application developed to simplify the process of storing, retrieving, and managing customer data. It bridges the gap between raw data storage and accessible user management through a streamlined web interface.

## 1.2 Purpose

- To digitalize and organize customer record management.
- To provide an efficient CRUD (Create, Read, Update, Delete) interface for administrative operations.
- To ensure reliable data persistence using a relational database (MySQL).

# 2. IDEATION PHASE

## 2.1 Problem Statement

Manual management of customer records leads to data redundancy, difficulty in searching, and high risk of data loss. Organizations require an automated, centralized system to maintain accuracy.

## 2.2 Empathy Map

- **THINKS**: How can I store customer data without manual errors? Is there a way to update details instantly?
- **FEELS**: Frustrated by slow, paper-based or unorganized file systems.
- **SAYS**: "We need a digital dashboard to view all customer interactions."
- **DOES**: Manually inputs data; uses basic spreadsheets; runs CRUD operations via the web interface.

## 2.3 Brainstorming

- **Data Entry**: Simplistic forms for data ingestion.
- **Visualization**: Listing customers in a tabular format for quick overview.
- **Data Integrity**: Using unique identifiers (IDs) for updates and deletions.

# 3. REQUIREMENT ANALYSIS

## 3.1 Customer Journey Map

- **Awareness**: User identifies the need for customer tracking.
- **Consideration**: Evaluating full-stack options (Node.js/MySQL).
- **Decision**: Selecting the current tech stack for performance.
- **Action**: Performing CRUD operations via the dashboard.

## 3.2 Solution Requirement

- **Functional**: Add customer, View customer list, Update name by ID, Delete records by ID.
- **Non-Functional**: Scalability, consistent data connection, and responsive UI.

### 3.3 Data Flow Diagram

Frontend (HTML/JS) &Fetch API & Express Server & MySQL Database.

### 3.4 Technology Stack

- **Frontend**: HTML, Bootstrap, JavaScript.
- **Backend**: Node.js, Express.js.
- **Database**: MySQL (mysql2 driver).

## 4. PROJECT DESIGN

### 4.1 Proposed Solution

A centralized Node.js server acts as the backbone, connecting a responsive frontend to a relational database, allowing for secure and structured data operations.

### 4.2 Solution Architecture

The application follows a standard **Client-Server architecture** where the client sends HTTP requests (GET, POST, PUT, DELETE) to the server, which then processes SQL queries to manage the database state.

## 5. PROJECT PLANNING & SCHEDULING

- **Phase 1**: Database schema design (db.sql).
- **Phase 2**: Backend API development (app.js).
- **Phase 3**: Frontend UI and Fetch integration (script.js).
- **Phase 4**: Testing and bug fixing.

## 6. FUNCTIONAL AND PERFORMANCE TESTING

- **Test 1**: Verify data persistence by adding a customer and checking the database.
- **Test 2**: Verify the Update functionality for changing customer names.
- **Test 3**: Verify the Delete functionality to ensure record removal.

## 7. RESULTS

*Developed a Customer Registry System using Node.js, Express.js, MySQL, HTML, CSS, and JavaScript to perform CRUD (Create, Read, Update, Delete) operations on customer data.*

## 8. ADVANTAGES & DISADVANTAGES

- **Advantages**: Lightweight, easy to deploy, uses standard web protocols, and efficient database management.
- **Disadvantages**: Requires local server setup; lacks advanced authentication in the current version.

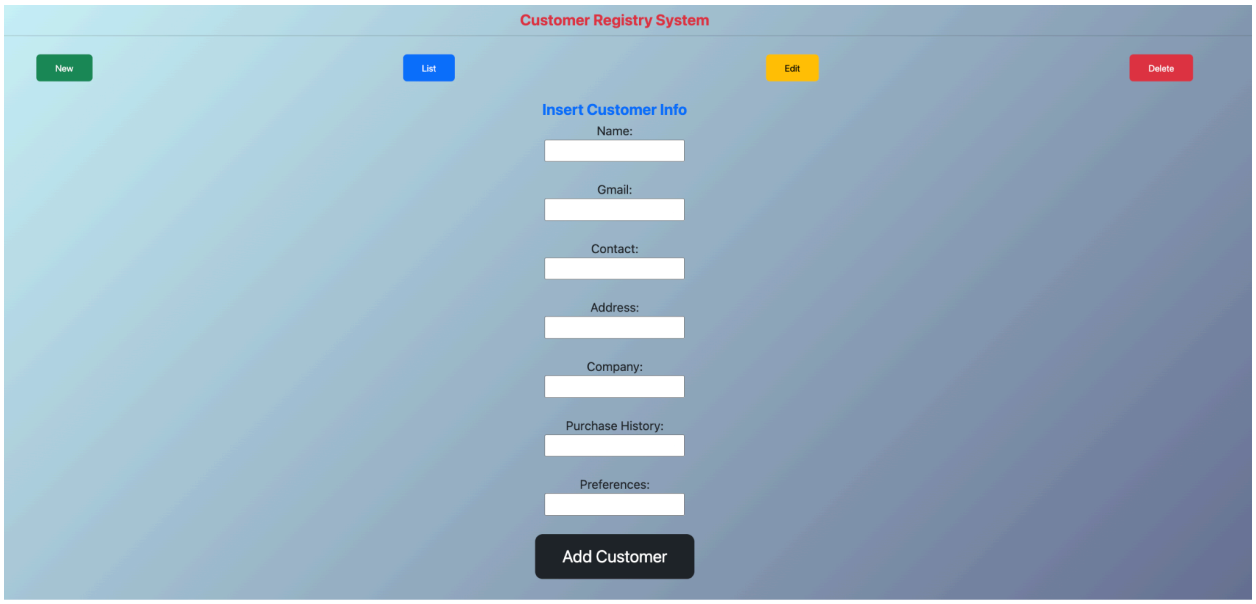
# 9. CONCLUSION

The Customer Registry System successfully implements a robust CRUD architecture. It provides a reliable foundation for small-scale customer data management and demonstrates effective integration between frontend and backend technologies.

# 10. FUTURE SCOPE

- **User Authentication**: Implementing JWT-based login.
- **Advanced Search**: Adding a real-time filter to search customers by name.
- **Export Data**: Adding functionality to export registry data to Excel or PDF.

### Main Page: Customer Registration Interface




### Overview

The **New Customer Registration** page serves as the primary entry point for the application. It provides an intuitive interface for administrators to onboard new customers into the registry system efficiently.

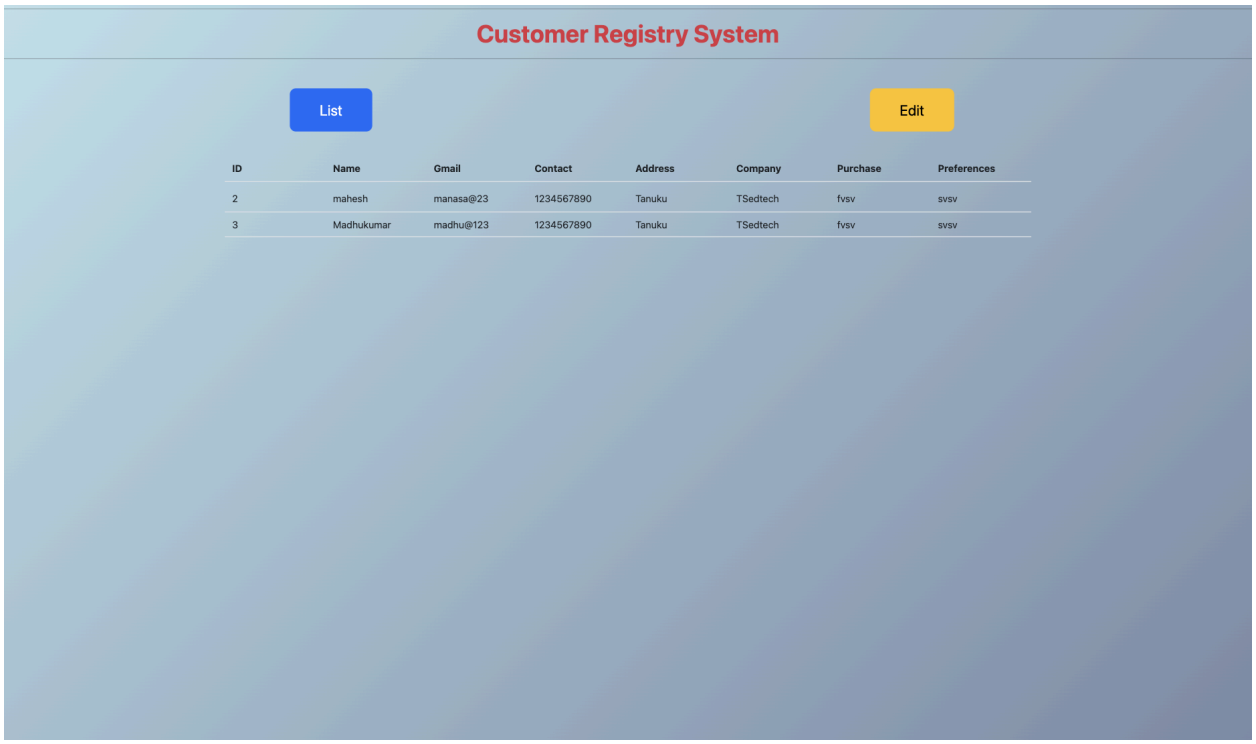
### Page Components

1. **Navigation Bar**:

- Contains four primary action buttons: **New**, **List**, **Edit**, and **Delete**. These allow the user to toggle between different modules of the Customer Registry System seamlessly.

2. **Input Form ("Insert Customer Info")**:
   - The form is designed to collect comprehensive customer data required for the registry. The fields include:
     - **Name**: Full name of the customer.
     - **Gmail**: Primary email address for communication.
     - **Contact**: Phone number for quick reach.
     - **Address**: Residence or billing location.
     - **Company**: Organization association.
     - **Purchase History**: Previous transaction details.
     - **Preferences**: Specific customer interests or choices.
3. **Submission Action**:
   - **"Add Customer" Button**: A primary action button that triggers the API call to save the provided information securely into the MySQL database.

## Customer Registry: Customer List View (Data Display)



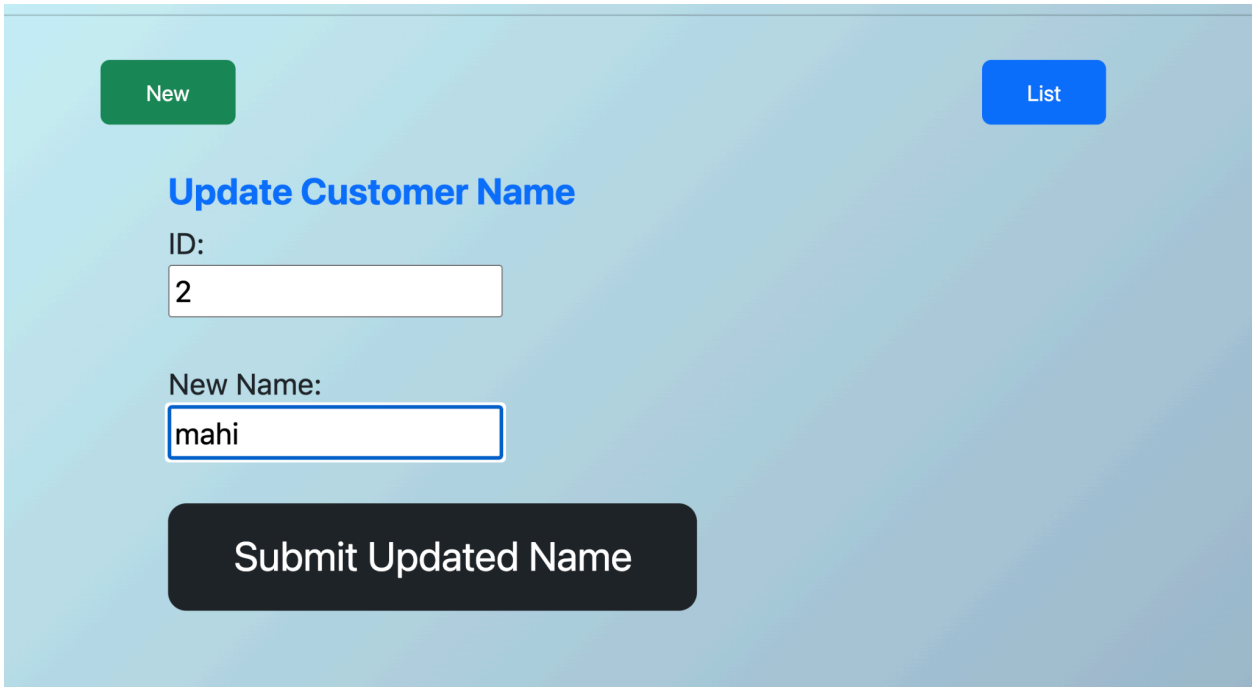
### Overview

The **Customer List View** page acts as the central dashboard for the application, displaying all registered customers currently stored in the MySQL database. It provides a clean, tabular summary for administrators to monitor the registry effectively.

### Page Components

1. **Navigation Controls**:
    - **List Button**: Refreshes the display to ensure the latest records are shown.
    - **Edit Button**: Provides direct access to update or modify existing customer information.
2. **Data Table**:
    - This section renders the data dynamically fetched from the /customers API endpoint.
    - **Key Fields Displayed**:
        - **ID**: The unique identifier for each customer.
        - **Name**: Registered customer name.
        - **Gmail**: Associated email contact.
        - **Contact**: Phone number.
        - **Address**: Customer location.
        - **Company**: Organizational details.
        - **Purchase/Preferences**: Insight into customer history and interests.

## Module: Customer Data Update (Edit)



### Overview

The **Update Customer Name** page provides the functionality to modify existing customer records without creating duplicate entries. It ensures that administrative data stays current by allowing updates based on the unique customer identification number.

### Page Components

1. **Navigation Bar**:
    - **New**: Redirects to the customer registration form.
    - **List**: Navigates back to the main customer registry table.
2. **Update Form ("Update Customer Name")**:
    - **ID Field**: A precise input field where the administrator enters the unique customer_id of the record that requires an update.
    - **New Name Field**: An input field to define the corrected or updated name for the selected customer.
3. **Submission Action**:
    - **"Submit Updated Name" Button**: Triggers a PUT request to the backend /update endpoint, which performs an SQL update operation in the MySQL database based on the provided ID.

## Module: Post-Update Verification

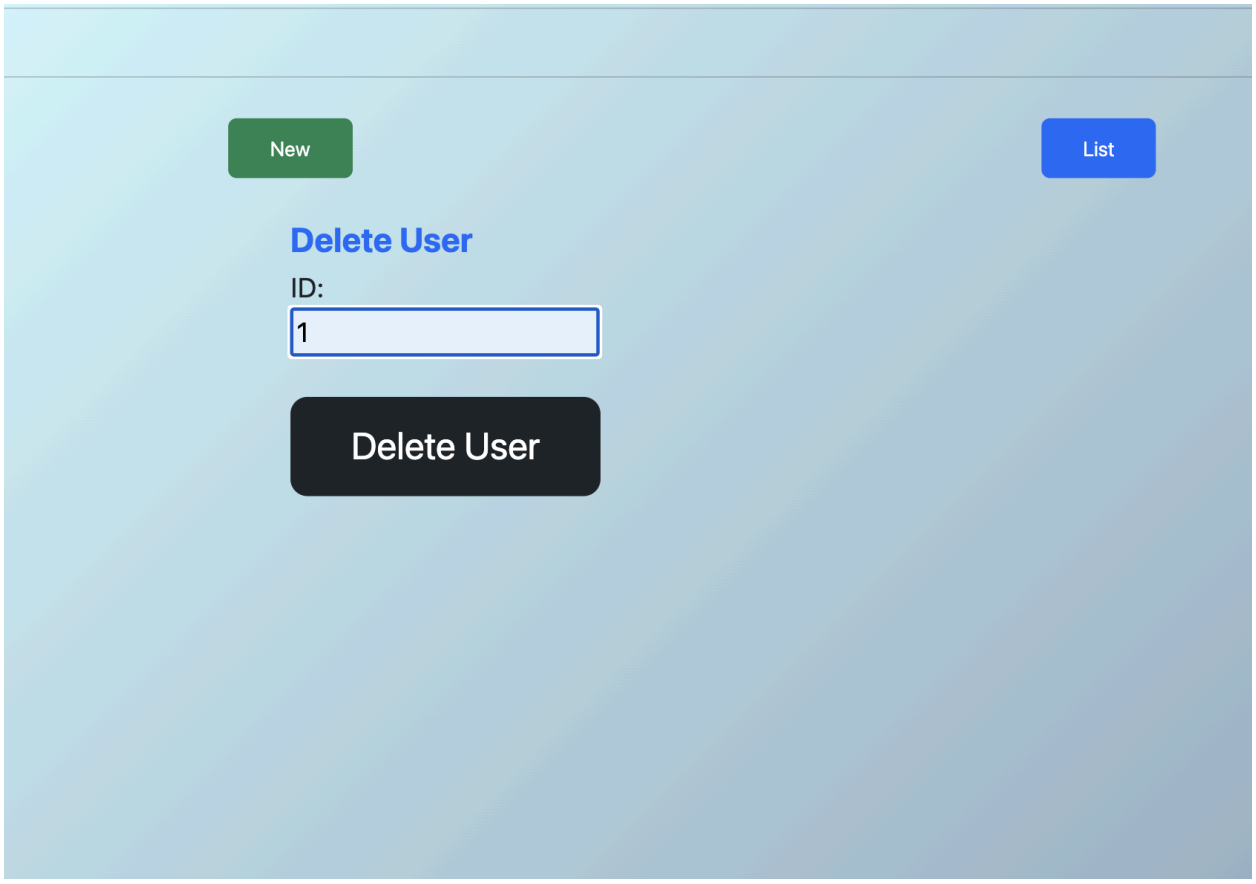

### Overview

Once the name update operation is successfully performed, the system automatically redirects or triggers a refresh to the **Customer List View**. This page acts as the verification module, allowing the administrator to confirm that the changes made in the database are accurately reflected in the registry.

### Key Observations

- **Data Consistency**: The table clearly displays the updated name ("mahi") associated with the specific customer_id (2).
- **Real-time Update**: The synchronization between the backend database (MySQL) and the frontend UI confirms that the PUT /update API endpoint is functioning as expected.
- **Verification**: This step validates the integrity of the data update process, ensuring that the system reliably handles state changes for existing customer records.

## Module: Customer Record Deletion



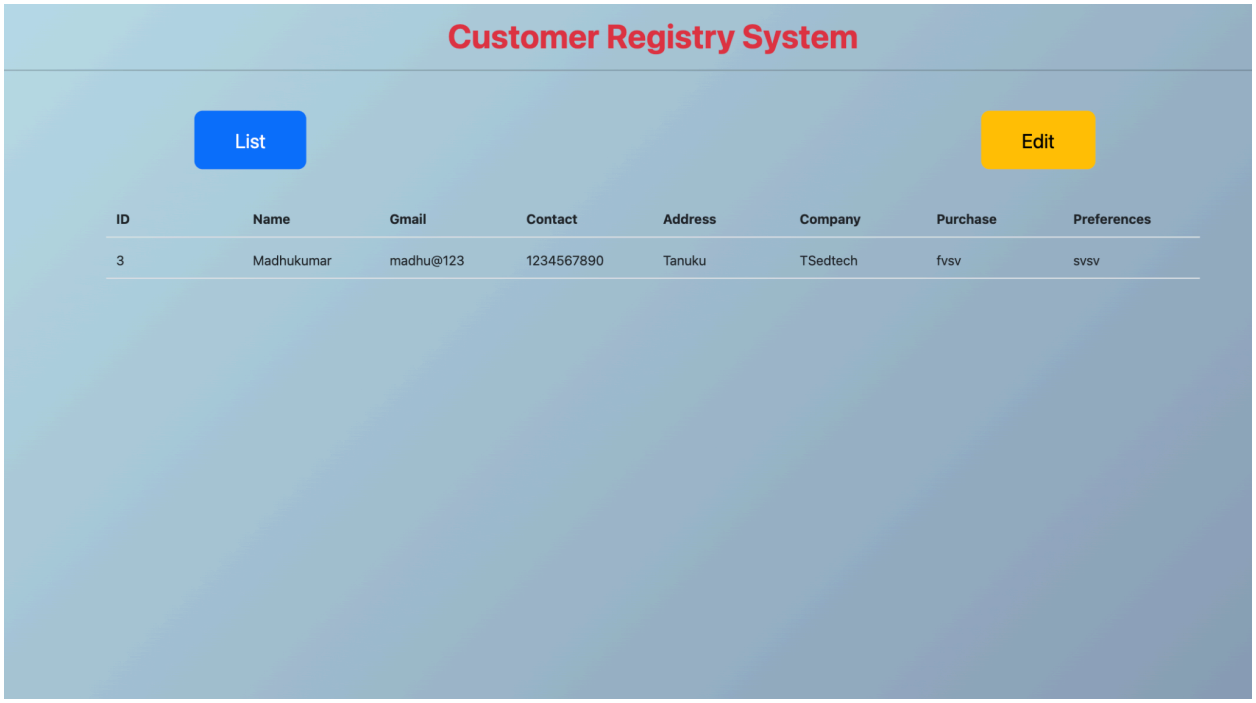

### Overview

The **Delete User** page provides a dedicated interface for administrators to remove customer records from the registry system that are no longer required. This module ensures database hygiene by facilitating the permanent removal of specific customer entries.

### Page Components

1. **Navigation Bar**:
    - **New**: Directs to the registration interface for creating new entries.
    - **List**: Navigates to the main registry table to view existing data.
2. **Delete Form ("Delete User")**:
    - **ID Field**: A specific input field where the administrator enters the unique customer_id of the record to be removed.
3. **Submission Action**:
    - **"Delete User" Button**: A high-contrast action button that triggers a DELETE request to the backend /delete endpoint, executing an SQL removal command in the database based on the provided ID.

## Module: Post-Deletion Verification




### Overview

After a successful delete operation, the system automatically refreshes the registry table to provide immediate visual feedback to the administrator. This page confirms that the targeted record (in this case, the record with ID: 1) has been permanently removed from the database.

### Key Observations

- **Data Synchronization**: The list view now only displays the remaining records (ID: 3), confirming that the backend DELETE operation was executed successfully in the MySQL database.
- **System Responsiveness**: The automatic re-fetching of the dataset ensures that the UI always represents the "Single Source of Truth" currently residing in the database.
- **Integrity**: The successful removal of the record without affecting other entries proves the robustness of the delete logic implemented in the application.

## Module: Database Schema & Management

```
mysql> use march;
Database changed
mysql> show tables;
+-----------------+
| Tables_in_march |
+-----------------+
| customers       |
+-----------------+
1 row in set (0.003 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| college            |
| information_schema |
| march              |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
6 rows in set (0.001 sec)

mysql> use march;
Database changed
mysql> show tables;
+-----------------+
| Tables_in_march |
+-----------------+
| customers       |
+-----------------+
1 row in set (0.002 sec)

mysql> select * from customers;
+-------------+------------+-----------+------------+---------+----------+-----------------+-------------+
| customer_id | name       | gmail     | contact    | address | company  | purchaseHistory | preferences |
+-------------+------------+-----------+------------+---------+----------+-----------------+-------------+
|           3 | Madhukumar | madhu@123 | 1234567890 | Tanuku  | TSedtech | fvsv            | svsv        |
+-------------+------------+-----------+------------+---------+----------+-----------------+-------------+
1 row in set (0.001 sec)
```

### Overview

The application uses **MySQL** as its relational database management system to ensure structured and persistent storage of customer information. The database is named march and contains a single primary table named customers which handles all CRUD operations.

### Table Schema: customers

The customers table is designed with the following structure to capture all necessary user details:

| Column Name | Data Type | Description |
|---|---|---|
| customer_id | INT | Primary Key, unique identifier for each customer |
| name | VARCHAR(100) | Full name of the customer |
| gmail | VARCHAR(100) | Email address |
| contact | VARCHAR(20) | Phone number |
| address | VARCHAR(255) | Residential or office address |
| company | VARCHAR(100) | Associated organization |
| purchaseHistory | VARCHAR(255) | Details of previous purchases |
| preferences | VARCHAR(255) | Customer-specific preferences |

**Management Verification**

- **Database Selection**: The system explicitly uses the march database to isolate the customer registry data.
- **Data Retrieval**: The terminal output verifies that a SELECT * query successfully fetches the stored records, confirming that the backend API is correctly communicating with the database layer.
- **Data Integrity**: By maintaining columns such as purchaseHistory and preferences, the database schema supports detailed customer profiling, which is essential for business analytics and personalized services.

```
const db = mysql.createConnection({

 host: "localhost",

 user: "root",

 password: "sanju@123",

 database: "march",

});
```

## Database Configuration & Connection

### Overview

The application establishes a secure connection to the MySQL database server using the mysql2 driver. This connection allows the Express.js server to execute SQL queries for all CRUD operations.

### Configuration Details

The database connection is configured with the following parameters to ensure proper communication between the Node.js backend and the MySQL server:

- **Host**: localhost (Default local development environment)
- **User**: root (Administrative database user)
- **Password**: sanju@123

- **Database**: march (The target database containing the customers table)